

Python을 이용한 파일 처리

01 파일 다루기 기초

목차 01 파일 다루기 기초

1. open, close
2. 읽기와 쓰기
3. tell, seek
4. with 문 사용하기
5. 예외 처리하기



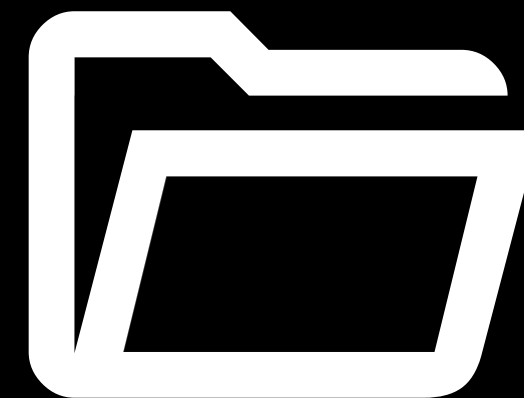
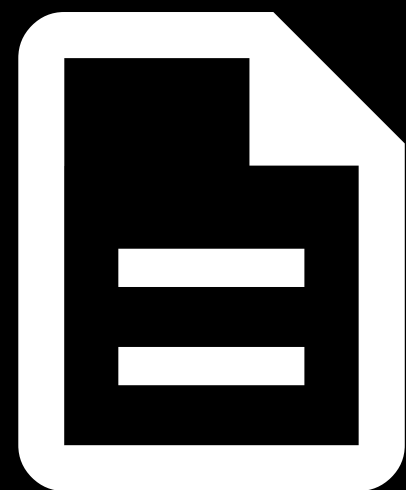
파일이란?

Python을 이용한 파일
처리

01 파일 다루기 기초

시작하기에 앞서

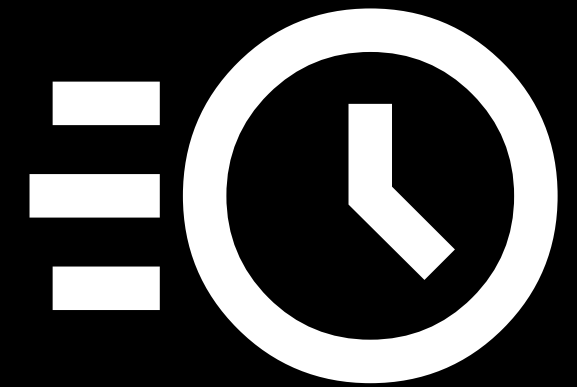
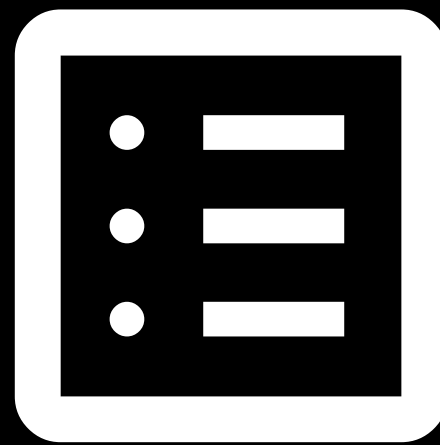
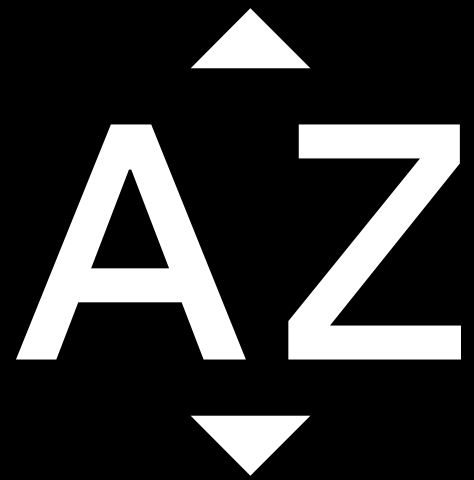
- 의미가 있는 정보를 구분하는 단위
- 크게 두 분류로 나눌 수 있음
 - 일반 파일
 - 폴더가 아닌 파일, 실행할 수 있는 파일이나 문서 및 압축 파일 등이 여기에 속함
 - 디렉터리
 - 폴더, 파일을 계층형 구조로 보관할 수 있도록 만듦





- 실행 가능 파일
 - 기계를 제어하는 내용이 담긴 파일, 사람이 읽을 수 없음
- 이진 파일
 - 특정한 방법을 통해서 확인 가능한 파일, 사람이 읽을 수 없음
 - 예: 압축파일(ZIP), PDF 문서, 이미지, 사운드
- 단순 텍스트
 - 특정한 형식 없이 작성된 파일, 일반적으로 .txt로 끝남, 사람이 읽을 수 있음
- 형식화된 텍스트
 - 특정 형식에 맞게 작성된 파일, 사람이 읽을 수 있으며 컴퓨터가 처리하도록 할 수 있음
 - 예: Python 파일, HTML, XML, CSV, JSON

- 많은 파일 이름 한 번에 변경하기
- 파일 자동으로 분류하여 폴더에 정리하기
- 파일에서 필요한 부분만 추출하기



1. open, close

```
f = open("filename.txt")
print(type(f))
# <class '_io.TextIOWrapper'>
f.close() # 꼭 수행 필요
```

- Python에서 파일을 여는 함수
- 파일 객체를 반환
 - 읽기 혹은 쓰기 작업이 가능한 객체
- 기본 사용법: **open(파일 이름)**
- 여러 가지 모드가 있으며 지정하지 않으면 읽기 전용으로 열림
 - 읽기 전용으로 여는 경우 파일이 존재하지 않으면 **FileNotFoundError** 발생
- 읽기 전용으로 파일을 쓸 수 없으며 쓰기 전용으로 파일을 읽을 수 없음
- 파일의 사용이 끝난 후에는 **close** 메서드를 호출하여 파일 닫는 작업을 수행해야 함



- **open** 함수 이용시 파일 이름 다음에, 혹은 명시적으로 **mode** 인자에 전달하는 방식으로 설정 가능
- 종류(사용 옵션)
 - **r**: 읽기 전용(기본값)
 - **w**: 쓰기 전용(파일이 존재하지 않으면 생성, 파일이 존재하면 기존 내용을 덮어씀)
 - **x**: 쓰기 전용(파일이 이미 존재하면 **FileExistsError** 발생)
 - **a**: 쓰기 전용(파일이 존재하지 않으면 생성, 파일이 존재하면 기존 내용 다음에 추가)
- 종류(처리 옵션)
 - **b**: 바이너리 모드
 - **t**: 텍스트 모드(기본값)
- 종류(추가 옵션)
 - **+**: 갱신(읽기 및 쓰기)



파일 열기 옵션 사용하기

Python을 이용한 파일
처리

01 파일 다루기 기초

1. open, close

```
f = open("file.txt")  
# open("file.txt", "rt") 혹은 open("file.txt", mode="rt")와 동일  
f.close()  
f = open("file.txt", "w")  
# 위 open 함수가 실행되면 파일의 모든 내용이 삭제됨  
f.write("Python")  
f.close()
```

```
f = open("file.txt", "a")  
f.write("Python")  
f.close()
```

```
f = open("file.txt", "r+")  
print(f.read())  
# Python  
f.close()
```

```
f = open("file.txt", "w+")  
# 위 open 함수가 실행되면 파일의 모든 내용이 삭제됨  
f.write("Python")  
f.seek(0) # 파일 객체의 처음 위치로 되돌아감  
print(f.read())  
# Python  
f.close()
```



- Python은 텍스트 모드로 **open** 함수 사용시 **encoding** 인자를 따로 전달하지 않으면 시스템 설정에 따라 임의의 인코딩을 사용할 수 있음
 - 특별한 경우가 아니라면 새 파일을 만들 때는 **encoding="utf-8"**을 통해 인코딩을 UTF-8으로 명시적으로 지정하는 것이 좋음
 - UTF-8: 존재하는 가능한 모든 문자를 표현하기 위한 인코딩
- 국내의 공공데이터포털 혹은 외국의 자료를 가져올 때는 UTF-8을 사용하지 않는 경우가 있음
 - 이러한 경우에는 인코딩을 따로 지정해야 함

```
f = open("file.txt", "w", encoding="utf-8")  
# 권장하는 파일 열기 방법: encoding을 명시적으로 지정  
f.close()
```

```
f = open("file_windows-1252.txt", encoding="windows-1252")  
# Windows-1252로 인코딩된 파일 열기  
f.close()  
  
f = open("file_euc-kr.txt", encoding="euc-kr")  
# EUC-KR로 인코딩된 파일 열기  
f.close()
```

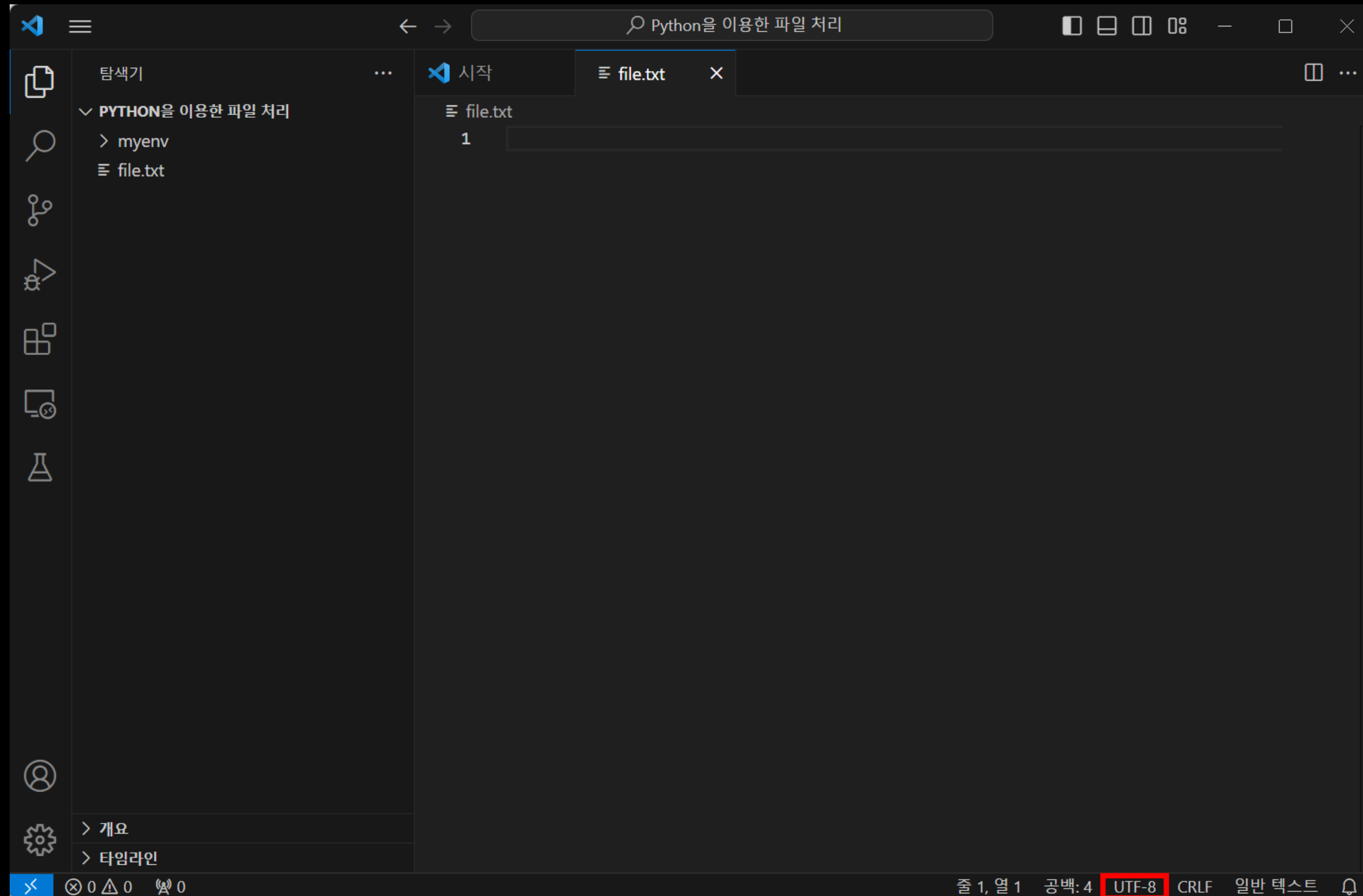


파일 인코딩 확인하기

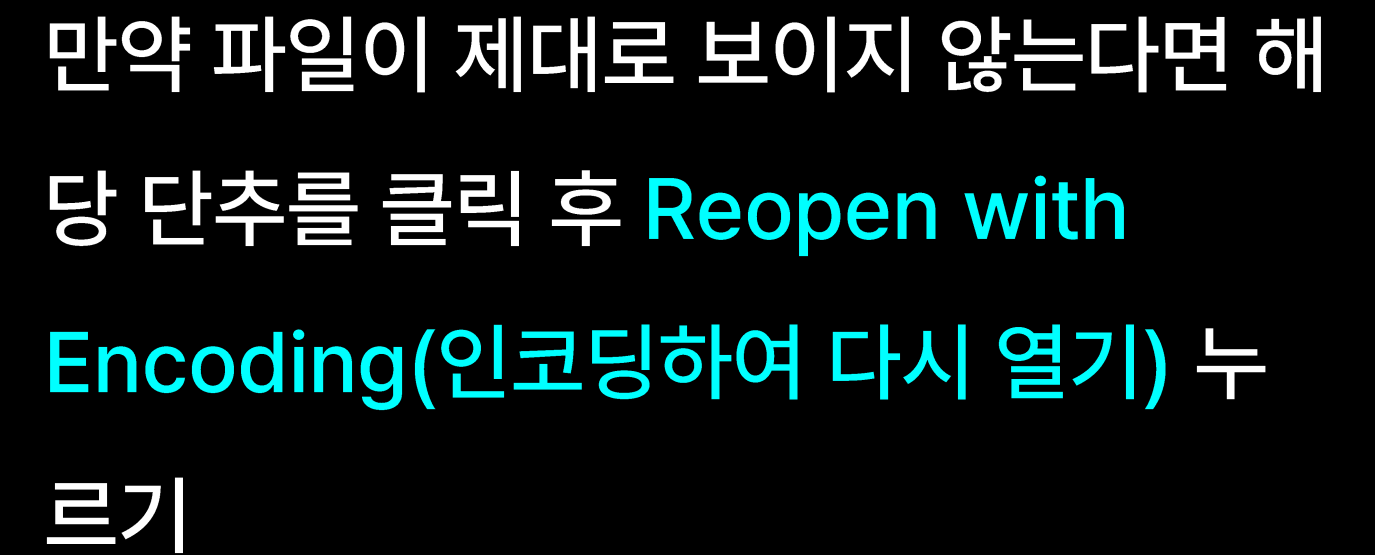
Python을 이용한 파일
처리

01 파일 다루기 기초

1. open, close



Visual Studio Code에서 인코딩을
확인하려는 파일 클릭 후 우측 하단을
확인



파일에 문제가 없는 경우 Visual Studio Code가 추측한 인코딩이 일반적으로 맞음

- 파일을 닫는 메서드
- 이상적으로 사용이 끝난 파일은 최대한 빨리 닫혀야 함
- **open** 메서드로 연 파일은 닫히기 전까지 잠김
 - 파일을 삭제하거나 이동하는 등의 동작이 불가능해짐
- 쓰기 모드로 파일을 썼다 하더라도 파일을 닫기 전까지는 실제로 기록하지 않을 수 있음
- 여러 파일이 닫히지 않고 열려 있으면 시스템의 자원을 많이 사용하게 됨

2. 읽기와 쓰기

파일 내용:

Monty
Python's
Flying
Circus

```
f = open("file.txt", encoding="utf-8")
print(f.read())
# Monty
# Python's
# Flying
# Circus
f.close()
```

- 파일 객체의 현재 위치에서 마지막까지의 내용을 읽는 메서드
- 줄 바꿈에 관계 없이 파일의 내용을 일정 크기 혹은 전체를 읽는 경우에 유용
- 괄호 안에 크기를 지정하여 전체 내용을 읽지 않고 부분만 읽을 수도 있음
 - 문자 단위(binary mode의 경우 byte 단위)
 - 한글의 경우 UTF-8 기준 3 bytes
 - 텍스트 모드에서는 1만 지정해도 한글이라면 3 bytes를 읽음

파일 내용:

Monty

Python's

Flying

Circus

```
f = open("file.txt", encoding="utf-8")
```

```
print(f.readline())
```

```
# Monty
```

```
#
```

```
f.close()
```

- 파일 객체의 현재 위치에서 하나의 줄을 읽는 메서드
- 파일 앞 부분의 몇 줄만 필요한 경우 유용함
- 필요하다면 괄호 안에 크기를 지정하여 고정된 길이의 문자열만 읽을 수 있음
 - 크기는 문자 단위(binary mode의 경우 byte 단위)
- 줄을 읽을 때 마지막 줄 바꿈 문자(**\n**)가 같이 붙게 됨
 - 따라서 일반적인 경우 문자열의 **rstrip** 메서드로 줄 바꿈 문자를 제거해야 함

파일 내용:

Monty

Python's

Flying

Circus

```
f = open("file.txt", encoding="utf-8")
```

```
print(f.readlines())
```

```
# ['Monty\n', "Python's\n", 'Flying\n', 'Circus']
```

- 파일 객체의 현재 위치에서 모든 줄을 읽어 리스트로 만드는 메서드
- 파일을 줄 단위로 가공하는데 유용함
- 필요하다면 괄호 안에 크기를 지정하여 처리할 수 있음
 - 크기는 문자 단위(binary mode의 경우 byte 단위)
- **readline**과 마찬가지로 문자열의 끝마다 줄 바꿈 문자가 붙는 점에 유의

```
f = open("file.txt", "w", encoding="utf-8")
f.write("Monty\n")
f.write("Python")
f.close()
```

```
f = open("file.txt", encoding="utf-8")
print(f.read())
# Monty
# Python
f.close()
```

- 파일의 현재 위치에서 문자열을 파일에 쓰는 메서드
- 줄 바꿈 문자(**\n**) 혹은 삼중 따옴표(**"""**) 표기를 이용하여 한 번에 여러 줄을 쓸 수도 있음

```
f = open("file.txt", "w", encoding="utf-8")
f.writelines(["Monty\n", "Python"])
f.close()
```

```
f = open("file.txt", encoding="utf-8")
print(f.read())
# Monty
# Python
f.close()
```

- 파일의 현재 위치에서 문자열 리스트를 파일에 쓰는 메서드
- **readlines** 메서드가 줄 바꿈 문자를 포함한 문자열의 리스트를 반환하는 것처럼 여러 줄을 쓰기 위해서는 리스트 안의 각 문자열 끝에 줄 바꿈 문자가 있어야 함

3. tell, seek

파일 내용:

Monty Python's Flying Circus

```
f = open("file.txt", encoding="utf-8")
```

```
print(f.read(3))
```

```
# Mon
```

```
print(f.tell())
```

```
# 3
```

```
f.close()
```

- 파일에서 현재 위치를 반환하는 메서드
- 처음 위치에서 바이트 단위로 몇 번째에 있는지 반환
- 파일 읽기와 쓰기는 모두 **tell** 메서드가 반환하는 위치에서 시작함
 - 예외: 옵션이 **a**라면 파일 쓰기는 항상 파일 끝에서 시작



커서 위치:

!Monty Python's Flying Circus

```
>>> f = open("file.txt", encoding="utf-8")
      f.read(3)
      f.close()
```

read 함수 실행 전 커서 위치

커서 위치:

Mon!ty Python's Flying Circus

```
      f = open("file.txt", encoding="utf-8")
>>> f.read(3)
      f.close()
```

read 함수 실행 후 커서 위치

파일 내용:

Monty Python's Flying Circus

```
f = open("file.txt", encoding="utf-8")
f.seek(3)
print(f.read())
# ty Python's Flying Circus
f.close()
```

- 파일의 현재 위치를 변경하는 메서드
- **seek(offset)** 형태로 사용
 - 파일의 처음에서 이동 시작
 - 현재 위치에서 이동하려면 **f.seek(f.tell() + offset)**과 같은 형태로 사용 가능

4. with 문 사용하기

- 진입 가능 객체(예: 파일 객체)를 한정된 구역에서 사용하도록 만들어주는 문장
- **with** 시작과 함께 객체가 생성되며 **with**를 벗어나면 객체가 자동으로 소멸됨
- 파일 객체의 경우 사용이 끝난 파일을 닫지 않는 실수를 기본적으로 방지
 - Python에서 파일을 다룰 때 특별한 이유가 없다면 항상 **with**를 사용하는 것이 좋음

```
# with 문을 사용하지 않을 때
f = open("file.txt", encoding="utf-8") # 파일 열림
content = f.read()
f.close() # 파일을 직접 닫아야 함
print(content)
```

```
# with 문을 사용할 때
with open("file.txt", encoding="utf-8") as f: # 파일 열림
    content = f.read()
# 알아서 파일 닫힘
print(content)
```



둘 이상의 파일 열기 비교

```
f1 = open("file1.txt", encoding="utf-8")
f2 = open("file2.txt", encoding="utf-8")
content1 = f1.read()
content2 = f2.read()
f1.close()
f2.close()
print(content1)
print(content2)
```

연 파일을 일일이 닫아야 하며 문장 간의 거리가 멀면
close를 하지 않는 실수를 일으킬 수 있음

```
with (
    open("file1.txt", encoding="utf-8") as f1,
    open("file2.txt", encoding="utf-8") as f2,
):
    content1 = f1.read()
    content2 = f2.read()
print(content1)
print(content2)
```

close에 대해 생각하지 않아도 되며 들여쓰기를 통
해 파일의 사용 위치를 시각적으로 확인할 수 있음

5. 예외 처리하기



```
1 / 0
```

```
ZeroDivisionError: division by zero
```

```
---
```

```
int("string")
```

```
ValueError: invalid literal for int() with base 10: 'string'
```

프로그램 논리상 실행이 불가능한 경우
에 발생하는 오류

예: 0으로 나누기, 숫자로 이루어지지
않은 문자열을 정수로 변환



```
try:  
    1 / 0  
except ZeroDivisionError:  
    print("0으로 나눌 수 없습니다.")
```

- 발생할 수 있는 예외에 대해서 "단순히 오류를 내고 종료하는 것" 이외의 동작을 정의하는 것
- try 문을 사용하여 정의
 - try에 예외가 발생할 수 있는 문장 작성
 - except에 발생할 수 있는 예외를 명시하고 동작 정의



```
>>> a, b = map(int, input("두 정수를 입력하세요: ").split())
두 정수를 입력하세요: 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: not enough values to unpack (expected 2, got 1)
```

- 프로그램이 작성자에 의해 완벽히 통제 가능하다면 예외 처리가 존재해서는 안됨
 - 예외가 발생하는 부분을 직접 고쳐야 함
- 실생활에서 사용하는 프로그램은 작성자에 의해 완벽한 통제가 불가능함
 - 사용자의 예상을 벗어나는 입력이 존재
 - 찾는 파일이 없을 수 있음
 - 서버와 통신한다면 서버가 응답하지 않을 수도 있음



- **try** 안에 예외가 발생할 수 있는 코드 작성
 - 사용자의 입력, 파일 열기 등
- **except**에 발생할 수 있는 예외와 동작 정의
 - **except**는 1 개 이상
 - 전반적인 예외 상황을 처리하고자 할 때는 **except Exception**을 사용
 - 예외 확인은 위에서 아래로 순서가 정해지므로 **Exception**은 항상 가장 마지막에 작성
- **else**에 예외가 발생하지 않았을 때만 실행할 동작 정의
 - **else**는 0 또는 한 개만 가능
- **finally**에 예외와 상관 없이 실행되어야 하는 동작 정의
 - **finally**는 0 또는 한 개만 가능



```
try:
    a, b = map(int, input("두 정수를 입력하세요: ").split())
except ValueError:
    print("입력이 올바르지 않습니다.")
except Exception:
    print("알 수 없는 오류가 발생했습니다.")
else:
    print(a + b)
finally:
    print("프로그램의 실행을 마쳤습니다.")
```